



MODULE : Le Cloud Computing & DevOps

Pr. F. Benabbou
Master DSBD
Faculté des Sciences Ben M'Sik Casablanca





TABLE OF CONTENTS

01 CLOUD COMPUTING

- Introduction générale
- **La Virtualisation**
- Les concepts de base du Cloud Computing
- Technologies émergentes du CC : Edge, Fog, ...
- Étude de cas et projet pratique

02 DevOps & Cloud

- Introduction générale
- La philosophie DeVops
- Version control systems (git)
- Continuous Integration CI
- Tests automatisés dans CI/CD
- Développement Continu CD
- Infrastructure en tant que Code (IaC)
- Surveillance et Journalisation
- Étude de cas et projet pratique

01

LE CLOUD COMPUTING





Virtualisation des applications

- Généralement les applications sont installées sur un poste de travail
- Mais il y a des Problèmes liés à une installation classique :
 - Non compatibilité entre différentes applications
 - Parfois impossibilité de faire cohabiter plusieurs versions de la même application
 - Problème de conflits de DLL (cas Windows : version de DLLs systèmes propre à une application)
 - Lenteur du déploiement des applications (installation/désinstallation),...
 - Impossibilité de les migrer d'un host vers un autre
 - Etc.

Virtualisation des application

- La virtualisation d'application est une solution qui consiste à séparer l'utilisation d'une application de son environnement matériels et logiciels d'exécution.
- Le but de la virtualisation d'applications est de réduire la complexité liée à la gestion et au déploiement des applications au sein d'un SI.



Avant Virtualisation



Après Virtualisation



Virtualisation des applications

- Plusieurs méthodes de virtualisation d'applications existent, on peut les grouper en 4 méthodes
 - Virtualisation de l'application en utilisant une machine virtuelle qui s'exécute sur le hôte.
 - Virtualisation de l'application dans un **espace utilisateur** sur le système d'exploitation hôte
 - La virtualisation d'application basée sur l'isolation de processus ou **conteneurs**
 - Virtualisation d'application basée sur le **cloud**



Virtualisation des App. en utilisant des MVs

- Cette approche consiste à exécuter l'application sur une VM qui est elle-même exécutée sur le système d'exploitation hôte.
- Dans ce cas c'est la VM qui fournit un environnement d'exécution isolé pour l'application, qui est séparé du système d'exploitation hôte et des autres applications.
- L'application peut utiliser son propre système d'exploitation et ses propres dépendances, qui sont installés dans la VM.
- La **VM** fournit également une couche **d'abstraction** pour l'application, qui permet de la rendre portable entre les différents systèmes d'exploitation et les différentes plateformes.
- Exemple : VMware Workstation, VirtualBox , QEMU, etc

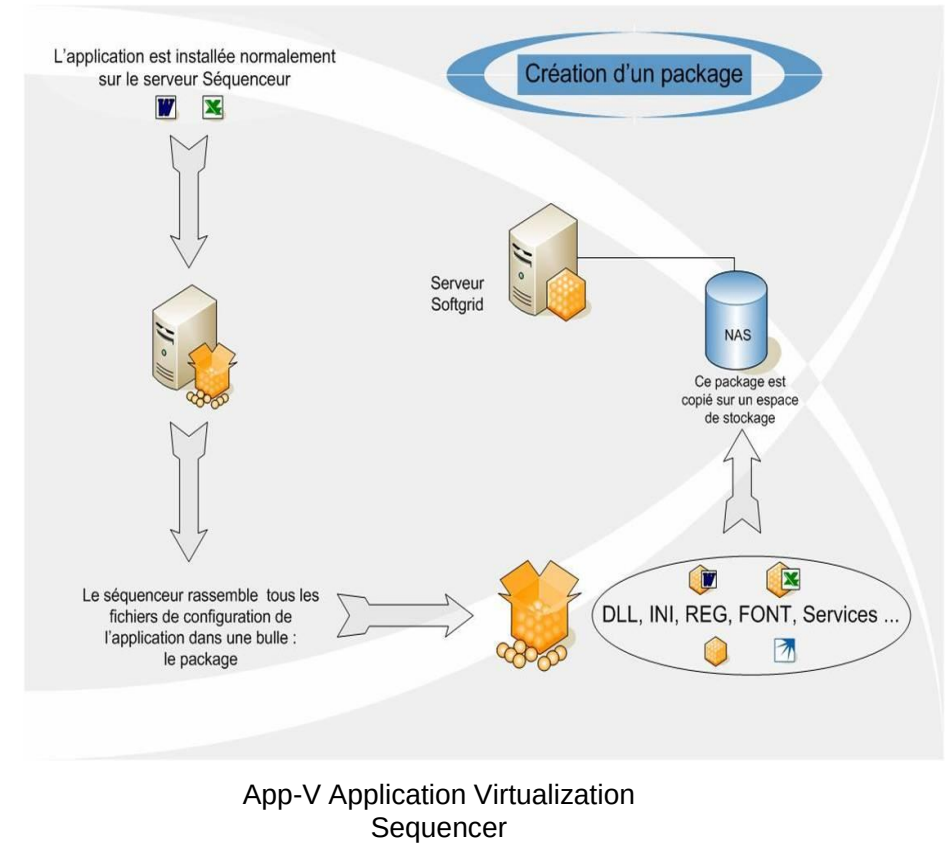


Virtualisation des App. dans un espace utilisateur

- Elle permet d'exécuter l'application dans un **espace utilisateur** indépendant et isolé du système d'exploitation de l'hôte et des autres applications installées sur le même système.
- Cette approche utilise une **couche d'abstraction** pour virtualiser l'application, qui sera **exécutée** directement sur **le système d'exploitation hôte**.
- Elle encapsule les applications dans un **package autonome** qui contient tout ce dont l'application a besoin pour fonctionner.
- Permet l'exécution d'applications sur des systèmes où elles ne peuvent pas être installées nativement.

Fonctionnement

- L'application virtuelle est créée sous forme d'un package par la **capture d'instantanés du système** physique ou elle est installée.
- On **emballe** l'application avec ses **dépendances**, éliminant les conflits entre applications.
- Les applications sont **packagées** sur un **serveur** et proposées à la demande.
- l'application est **exécutée** par le **poste de travail**, avec ses propres ressources.





Virtualisation des App. dans un espace utilisateur

- Turbo Studio, VMware ThinApp, Microsoft App-V, et Cameyo permettent de créer des packages d'applications portables qui peuvent être déployés sur des systèmes Windows sans installation.
- Limites :
 - La virtualisation des applications sur le système d'exploitation hôte est souvent limitée à un système **d'exploitation spécifique**
 - Certaines applications peuvent être **incompatibles** avec ces logiciels
 - Certaines applications **ne peuvent pas être virtualisées** par ces logiciels : par exemple ThinApp ne peut pas virtualiser les antivirus, pilotes d'imprimantes et scanner, applications nécessitant des pilotes de périphériques, certains VPN, etc.).
 - Il faut disposer d'une machine physique ou virtuelle propre à utiliser comme cible pour le "**séquençage**" (App-V) ou la "capture" (ThinApp).



Virtualisation basée sur les conteneurs

- Aussi appelé virtualisation basé sur **l'isolation des processus**
- Elle consiste à isoler les environnements d'exécution d'applications, le tout étant géré par un **unique noyau** celui du host.
- Bien que ce type de virtualisation n'est pas spécifique à Linux, elle repose sur les **fonctionnalités** de virtualisation du **noyau Linux** pour isoler les processus les uns des autres.
- L'isolation de processus est réalisée principalement grâce aux fonctionnalités du noyau Linux suivantes :
 - les espaces de noms (**namespace**)
 - les contrôles de groupe de processus (**cgroups**)
 - Changer le répertoire racine : **chroot**.



Les espaces de noms (namespaces)

- Les **namespaces** sont un mécanisme qui permet d'isoler les ressources système entre différents **processus** ou **groupes de processus**.
- Les **namespaces** créent un **contexte système isolé** pour le processus **ciblé** afin qu'il ne puisse pas voir ou accéder aux ressources (connections réseaux, fichiers, etc.) des autres processus ou groupe de processus
- Ce mécanisme va créer des **instances virtuelles** des ressources système, telles que les **UID**, **système** de fichiers, **PID**, le **réseau**, la **liste** des processus, les **utilisateurs**, les systèmes de communication interprocessus (**IPC**), etc.
- **Par exemple** : chaque groupe de processus a sa propre instance virtuelle du système de fichier permettant de monter/démonter les volumes sans incidence pour l'hôte ou les autres processus.



Les espaces de noms (namespaces)

- Dans le noyau Linux, il existe différents types d'espaces de noms avec leurs propres propriétés :
 - **user namespace** namespace possède son propre ensemble d'ID d'utilisateurs et d'ID de groupes à attribuer aux processus. Cela permet à un processus d'avoir le privilège root dans son espace de noms utilisateur sans l'avoir dans d'autres espaces de noms utilisateur.
 - **Process ID (PID)** namespace qui attribue aux processus un ensemble de PID indépendant de l'ensemble des PID des autres espaces de noms.
 - **Network** namespace possède sa propre table de routage privée, son ensemble d'adresses IP, sa liste de sockets, sa table de suivi des connexions, son pare-feu et d'autres ressources liées au réseau.
 - **Mount** namespace possède une liste indépendante de points de montage. Cela signifie qu'on peut monter et démonter des systèmes de fichiers dans un espace de noms de montage sans affecter le système de fichiers hôte.
 - **Interprocess communication (IPC)** namespace possède ses propres ressources IPC, ses propres files d'attente de messages pour communiquer.
 - **UNIX Time-Sharing (UTS) namespace** permet à chaque processus d'avoir son propre nom d'hôte (hostname) et son propre nom de domaine, qui peuvent être définis **indépendamment** du nom d'hôte et du nom de domaine réels du système.

Ubuntu 64-bit-1 - VMware Workstation

File Edit View VM Tabs Help

library

Type here to search

My Computer

- Ubuntu 64-bit-1
- Ubuntu 64-bit-2
- Windows 7 x64
- Windows Server 2012
- VMware ESXi 7 one
- VMware ESXi 7 two
- VCSA 6.7
- vcsa last
- mininet
- SDN-controller

SDN-controller mininet vcsa last Ubuntu 64-bit-2 Ubuntu 64-bit-1

Terminal

root@admin23-virtual-machine: ~

**UTS permet l'isolation du Nom d'Hôte et son domaine
unshare permet de créer un nouveau namespace et d'y
exécuter une commande ici c'est le shell**

Usage:
unshare [options] <program> [args...]

Options :
-h, --help usage information (this)
-m, --mount unshare mounts namespace
-u, --uts unshare UTS namespace (hostname etc)
-i, --ipc unshare System V IPC namespace
-n, --net unshare network namespace

Isoler et modifier le nom d'hôte (namespace UTS)

```
admin23@admin23-virtual-machine:~$ sudo unshare --uts bash
root@admin23-virtual-machine:~# hostname test-namespace
root@admin23-virtual-machine:~# hostname
test-namespace
root@admin23-virtual-machine:~#
```

**1-Lancer un shell dans un nouvel espace de noms
2-Changer le nom d'hôte dans ce nouveau namespace
3-Vérifier le nouveau nom d'hôte dans cet espace de noms
4-Quit et vérifier le nom du hôte**

admin23@admin23-virtual-machine: ~

```
root@admin23-virtual-machine:~# exit
exit
admin23@admin23-virtual-machine:~$ hostname
admin23-virtual-machine
admin23@admin23-virtual-machine:~$
```

4- Vérifier le nom Hôte dans le shell principal

to direct input to this VM, move the mouse pointer inside or press Ctrl+G.



Les contrôles de groupe de processus (cgroups)

- Un groupe de contrôle est une fonctionnalité du noyau Linux qui permet de **gérer, limiter, surveiller et isoler l'utilisation des ressources** (CPU, mémoire, E/S disque, réseau, etc.) d'un ensemble de processus.
- Ce mécanisme **empêche** un groupe de processus **d'utiliser les ressources** des autres groupe ou celle du système hôte.
- Les cgroups permettent de **contrôler la quantité d'une ressource** (CPU, mémoire, réseau et E/S disque) à laquelle un processus ou un ensemble de processus peut accéder ou qu'il peut utiliser.
- Les cgroups sont un élément clé dans un *environnement partagé* comme celui des conteneurs car il y a souvent plusieurs processus en cours d'exécution dans un conteneur qu'il faut contrôler l'ensemble.



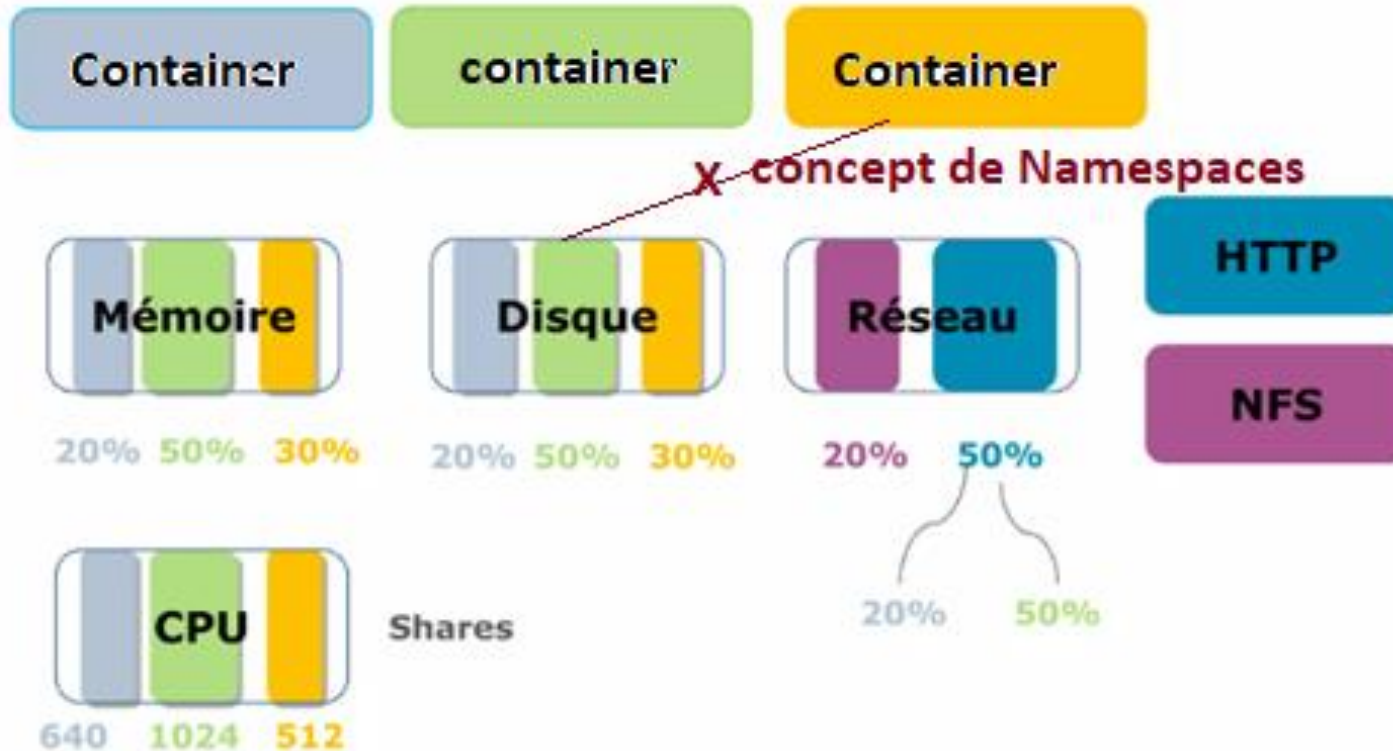
Les contrôles de groupe de processus (cgroups)

- Les Cgroups offrent les fonctionnalités suivantes :
 - **Limitation** des ressources permet de configurer un cgroup pour limiter la quantité d'une ressource particulière (mémoire ou CPU, par exemple) qu'un processus peut utiliser.
 - **Priorité** permet de contrôler la quantité d'une ressource (CPU, disque ou réseau) qu'un processus peut utiliser par rapport aux processus d'un autre cgroup lorsqu'il y a conflit de ressources.
 - **Surveillance** : Les limites de ressources sont surveillées et signalées au niveau du cgroup.
 - **Contrôle** : permet de modifier l'état (gelé, arrêté ou redémarré) de tous les processus d'un cgroup.



Cgroup et Namespaces

Concept de cgroup



On peut limiter le temps CPU max en nombre de milliseconde



Changer le répertoire racine

- La commande **chroot** permet de changer le répertoire racine vers un nouvel emplacement dans le système de fichier.
- Après avoir modifié la racine, le processus ne peut plus accéder aux fichiers ou aux utilitaires qui sont **plus haut dans la hiérarchie** des fichiers.
- C'est une commande qui prend comme unique argument un répertoire et y **fixe la racine** du système de fichiers pour le **processus qui l'invoque**.
- Ainsi on peut **emprisonner** l'exécution des processus dans son contexte.

```
sudo chroot /home/admin23/chroot_test /bin/ls
```

/home/admin23/chroot_test devient la **nouvelle racine** /
/bin/ls affiche en réalité le contenu de /home/admin23/chroot_test



Deux approches de conteneurs

■ Approche Système

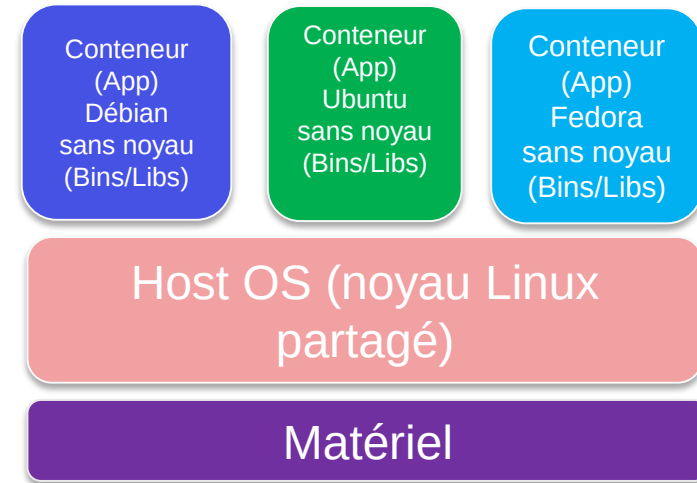
- OpenVZ.
- Linux Containers (LXC).

■ Approche Processus

- Docker.

Conteneur OpenVZ

- OpenVZ est une solution de virtualisation Open Source développée par Parallels Virtuozzo, est bien antérieur à LXC.
- Il utilise la même logique que celle de LXC sur le principe général, mais avec une différence technique fondamentale sur la nature du noyau.
 - OpenVZ a **intégré** les **espaces** de **noms**, les **cgroup** et les **chroots**, pour isoler les conteneurs les uns des autres et du système hôte.
 - OpenVZ utilise le noyau de l'hôte,
 - Ce noyau hôte (le noyau "OpenVZ") a été modifié pour gérer l'isolation et les ressources, alors que LXC s'appuie sur ce qui existe déjà nativement dans Linux.
 - Tous les conteneurs utilisent exactement la même version du noyau que l'hôte.
- Ces conteneurs sont appelés des **Virtual Private Servers** (VPS).
- Il n'y a **aucune charge** supplémentaire due a une **émulation**



Conteneur OpenVZ



- Chaque conteneur dispose de ses propres fichiers, utilisateurs, processus et interfaces réseau de manière isolée.
- Le système impose des quotas stricts sur la mémoire et le processeur pour garantir les ressources de chaque environnement.
- Le partage du noyau limite l'usage aux distributions Linux et exclut l'installation de Windows ou BSD.
- L'architecture permet d'exécuter simultanément des centaines de conteneurs sur un seul serveur physique.
- L'utilitaire vzctl gère le cycle de vie des instances et permet leur migration à chaud sans interruption.
- Il y a large gamme de templates pour créer des VPS (Debian, Ubuntu, CentOS, etc.).
- Cependant, à chaque nouvelle version officielle du noyau Linux, l'équipe d'OpenVZ devait réécrire ses patches pour les adapter, ce qui créait un retard important et une grande difficulté de maintenance.
- Cette solution a progressivement été abandonnée au profit de LXC et Docker

LXC (Linux Containers)



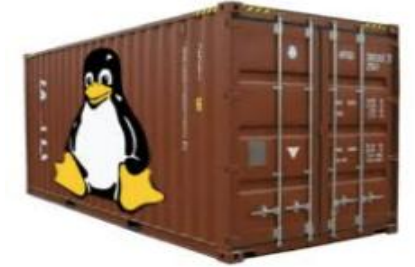
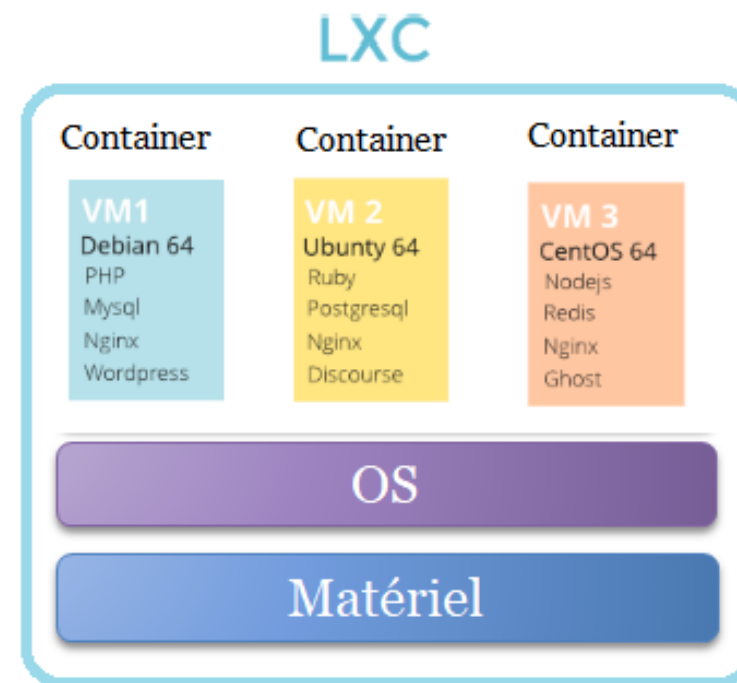
- C'est une technologie **open source** de virtualisation au niveau du système d'exploitation qui permet la création et l'exécution de plusieurs environnements virtuels Linux isolés (VE) appelés **conteneurs** sur le noyau Linux.
- Les **conteneurs** LXC **partagent les ressources du noyau** de la machine physique de l'hôte, ce qui les rend plus légers et plus rapides que les machines virtuelles.
- LXC se base sur les **cgroups** et des **namespaces** du noyau Linux afin de prendre en charge des environnements OS virtualisés légers
- Les conteneurs peuvent être utilisés soit pour cloisonner des **applications** spécifiques, soit pour émuler tout l'**hôte**.

Virtualisation Linux Containers (LXC)



LXC

- Il est possible de se connecter à un conteneur LXC, de le traiter comme un système d'exploitation, d'y installer une ou plusieurs applications et des services.
- Le système d'exploitation invité n'a pas son propre noyau mais il utilise le **noyau que l'hôte**.
- Cependant, le conteneur LXC possède sa propre arborescence de fichiers qui contient tous les outils d'une distribution Linux classique
- Un conteneur LXC est conçu pour être un conteneur de système (System Container).
- Il ne se contente pas de lancer une seule application, il simule un environnement complet.



Virtualisation Linux Containers (LXC)



- Les conteneurs LXC **ne sont pas des machines virtuelles** dans le sens traditionnel du terme, il n'y a pas d'hyperviseur et le matériel n'est pas virtualisé.
- Un conteneur apporte une virtualisation de l'environnement d'exécution (processeur, mémoire vive, réseau, système de fichier...) et non pas de la machine physique
- LXC fournit des modèles de conteneurs pour les systèmes d'exploitation de base et un ensemble complet d'outils pour la gestion du cycle de vie des conteneurs.
- Il prend en charge d'autres formats de conteneurs tels que les conteneurs **Docker** et les conteneurs **OCI** (Open Container Initiative).

OpenVZ vs LXC



OpenVZ

- **OpenVZ**, nécessite un noyau modifié
- OpenVZ est conçu pour fonctionner avec des systèmes invités qui utilisent la même famille de noyau que l'hôte.
- OpenVZ n'est pas compatible avec les technologies de conteneurs tels que Docker ou LXC.
- Cela peut limiter la portabilité des applications entre différents environnements de conteneurs.



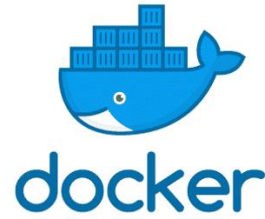
LXC

LXC

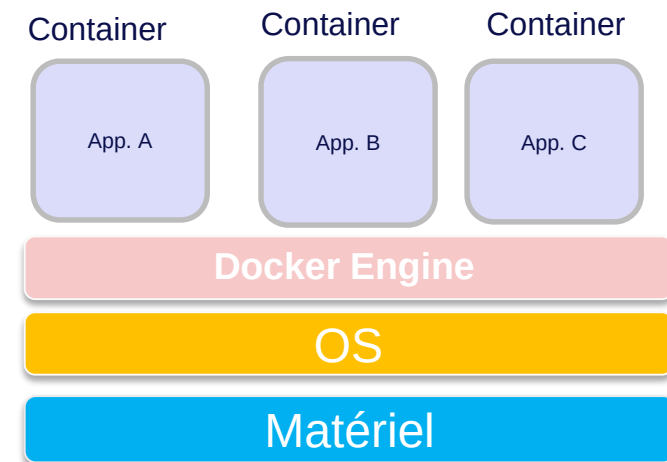
- **LXC** fonctionne avec le noyau Linux standard du hôte
- il peut être utilisé sur toute distribution Linux sans nécessiter de modifications au noyau.
- LXC prend en charge d'autres formats de conteneurs tels que les conteneurs Docker et les conteneurs OCI.



Docker



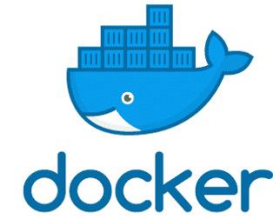
- Docker est un logiciel libre de virtualisation des applications basé sur l'isolation de processus.
- C'est une plateforme open source qui automatise le déploiement, la gestion et l'exécution d'applications à l'intérieur de conteneurs logiciels.
- Les conteneurs Docker encapsulent une application avec son environnement d'exécution et toutes ses dépendances, ce qui permet de les exécuter de manière cohérente et portable sur n'importe quel environnement compatible Docker,
- En théorie, les conteneurs Docker sont conçus pour contenir une seule application ou un service spécifique.



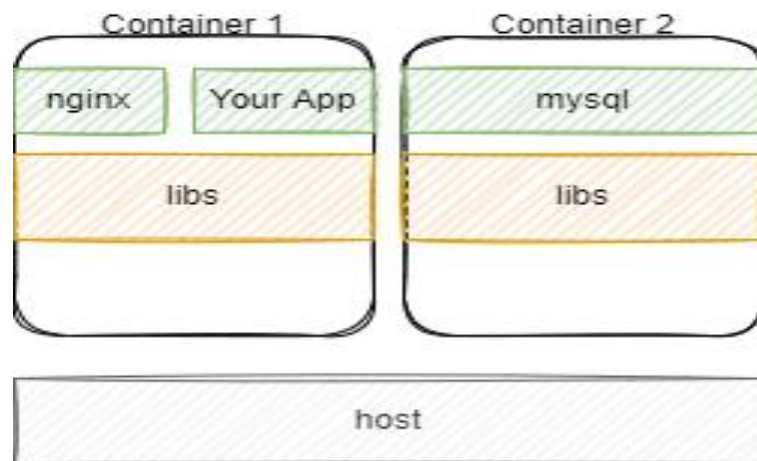
Le rôle de l'isolateur est assuré par Docker engine



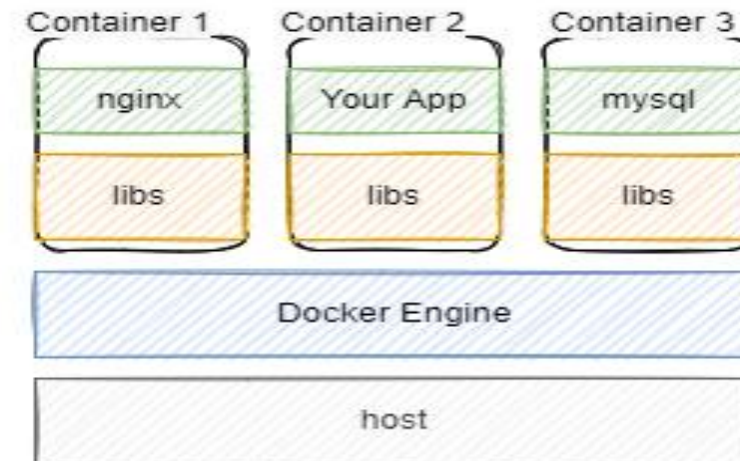
LXC vs Docker



- Docker a été initialement développé en utilisant les conteneurs LXC, mais il a évolué en adoptant une autre technologie de virtualisation appelée **libcontainer**.
- Cependant, Docker continue d'utiliser LXC pour certaines fonctionnalités telles que l'isolation des ressources et des processus.
- Le Docker Engine permet de créer, exécuter et gérer des conteneurs Docker
- Docker utilise une architecture client-serveur qui se base sur l'API REST.

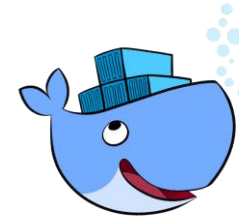
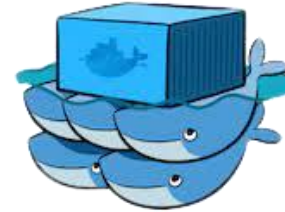
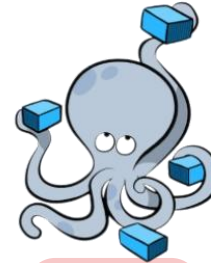
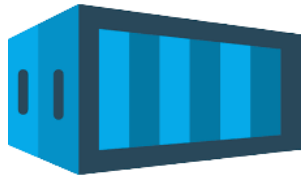
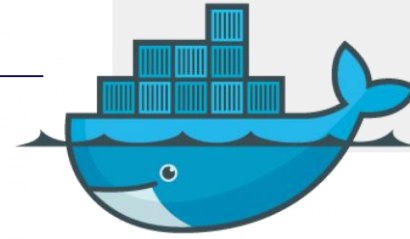


LXC



Docker

Concepts Clés



1

2

3

4

5

6

7



Conteneur

Image

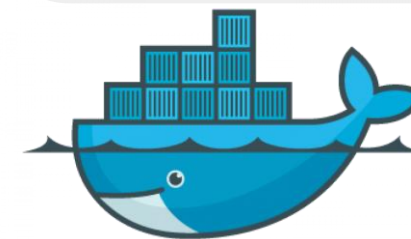
Dockerfile

Registre
Docker

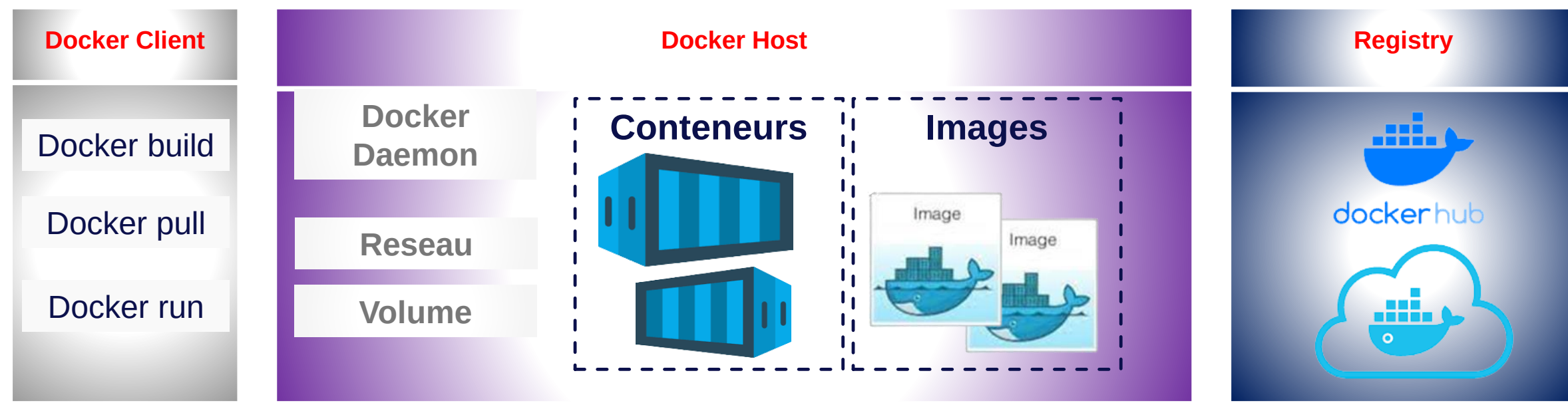
Docker
compose

Docker
Swarm

Docker Engine



Architecture Docker



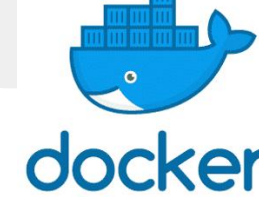
Docker client

- Le Docker Client, c'est l'interface qui permet de communiquer avec docker.

- CLI pour Linux
- WSL pour windows

- Voici les commandes les plus utilisées

- **docker pull** : permet de télécharger une image depuis un registre Docker public ou privé comme Docker Hub



Docker build

Docker pull

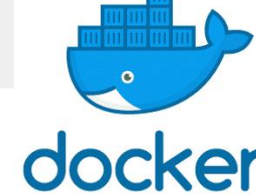
Docker run

```
C:\Windows\System32>docker pull python
Using default tag: latest
latest: Pulling from library/python
1227bf08bd42: Download complete
b1741a62ccee: Download complete
281b80c799de: Download complete
02e37abc533a: Download complete
378c64c44580: Download complete
15f14138abe4: Download complete
add107facb26: Download complete
Digest: sha256:6d58c1a9444bc2664f0fa20c43a592fcd62698eb9a9c32257516538a2746c19a
Status: Downloaded newer image for python:latest
docker.io/library/python:latest
```

```
C:\Windows\System32>docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
python	latest	6d58c1a9444b	9 days ago	1.61GB
localhost:5000/firstapp	local	3ff46c40d2e1	12 months ago	373MB
firstapp	latest	3ff46c40d2e1	12 months ago	373MB
faouziabenabbou/firstapp	latest	a9f050b924e6	13 months ago	369MB
faouziabenabbou/firstappv0.1	latest	a9f050b924e6	13 months ago	369MB
gcr.io/k8s-minikube/kicbase	v0.0.45	e7c9bc3bc515	16 months ago	1.81GB
gcr.io/k8s-minikube/kicbase	<none>	81df28859520	16 months ago	1.81GB
registry	2	543dade69668	2 years ago	37.2MB
hello-world	latest	305243c73457	2 years ago	24.4kB

Docker client



Docker build

Docker pull

Docker run

■ Voici les commandes les plus utilisées

- **docker build** : Construire une image Docker à partir d'un Dockerfile.
 - « . » contexte de build
 - -t permet d'associer un tag votre image
 - Dockerfile fichier de configuration de l'image

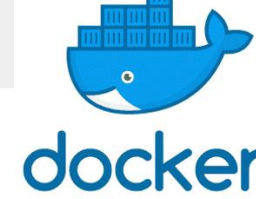
```
C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>docker build -t firstapp:2.0 -f Dockerfile .
[+] Building 17.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 164B
=> [internal] load metadata for docker.io/library/node:current-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:current-slim@sha256:b20f82c2332a22609edb43cdfd6b16cbb199a6e6fd5cf8d5ef85b43faab43144
=> => resolve docker.io/library/node:current-slim@sha256:b20f82c2332a22609edb43cdfd6b16cbb199a6e6fd5cf8d5ef85b43faab43144
=> => sha256:63de5d6fc7b3e6d3211e19851ff2cea7941bb939cc2eede963d142a0e56210e3 446B / 446B
=> => sha256:e102b4372b0ab274d8efc70ae291f72ba47c18ad93f8fc024a9560744cce61ac 1.71MB / 1.71MB
=> => sha256:8026c6400dcf42d71b26b551939a5afc0c901a298849dacc58b0c6c6205f4989 47.35MB / 47.35MB
=> => sha256:38ca042e42a0636347e2ea9e1685a12bef5c0ed650fe0f50addf5f0bab358631 3.31kB / 3.31kB
=> => sha256:0347dcb76707f7d71a7c0b3a5f4a63b97cdd9923e637e67ad65b3b2d4ba05942 28.23MB / 28.23MB
=> => extracting sha256:0347dcb76707f7d71a7c0b3a5f4a63b97cdd9923e637e67ad65b3b2d4ba05942
=> => extracting sha256:38ca042e42a0636347e2ea9e1685a12bef5c0ed650fe0f50addf5f0bab358631
=> => extracting sha256:8026c6400dcf42d71b26b551939a5afc0c901a298849dacc58b0c6c6205f4989
=> => extracting sha256:e102b4372b0ab274d8efc70ae291f72ba47c18ad93f8fc024a9560744cce61ac
=> => extracting sha256:63de5d6fc7b3e6d3211e19851ff2cea7941bb939cc2eede963d142a0e56210e3
=> [internal] load build context
=> => transferring context: 148.27kB
=> [2/5] WORKDIR /usr/src/app
=> [3/5] COPY package.json .
=> [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:a0faa14c1b52ee71350b250223bb90f070263b19ac4ccbd3bda6b72bae611743
=> => exporting config sha256:754c490d82970c96f69a655697284fe6b780f014c4e431253df1d2d1dc123aab
=> => exporting attestation manifest sha256:96db5759dc7245080aaef47aa12a0621f426cf381a69fa80c874a33ae3fe1ef7
=> => exporting manifest list sha256:0f24ad7c43187a8e8105d3974eea5fe4fa31d668f462888d2a6bb3957932f9df
=> => naming to docker.io/library/firstapp:2.0
=> => unpacking to docker.io/library/firstapp:2.0
```

View build details: [docker-desktop://dashboard/build/desktop-linux/desktop-linux/wrs9efedr1mbv7o0wdt45akxd](#)

```
C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>docker build -t firstapp:2.0 -f Dockerfile .
```

```
C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
firstapp             2.0        0f24ad7c4318  2 minutes ago  370MB
```

Docker client



Docker build

Docker pull

Docker run

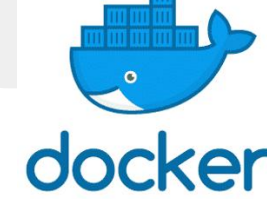
- Voici les commandes les plus utilisées
 - **docker run** : permet de démarrer un conteneur à partir d'une image.

```
C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>docker run -d -p 8180:8080 firstapp
28abe5127dbc6dc6ba7116635233427e1de68bb756c95139a671ca522da9c4f8

C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>docker container ls
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                    NAMES
28abe5127dbc   firstapp   "docker-entrypoint.s..." 30 seconds ago Up 30 seconds  0.0.0.0:8180->8080/tcp    wonderful_golick

C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>
```


Docker client



Docker build

Docker pull

Docker run

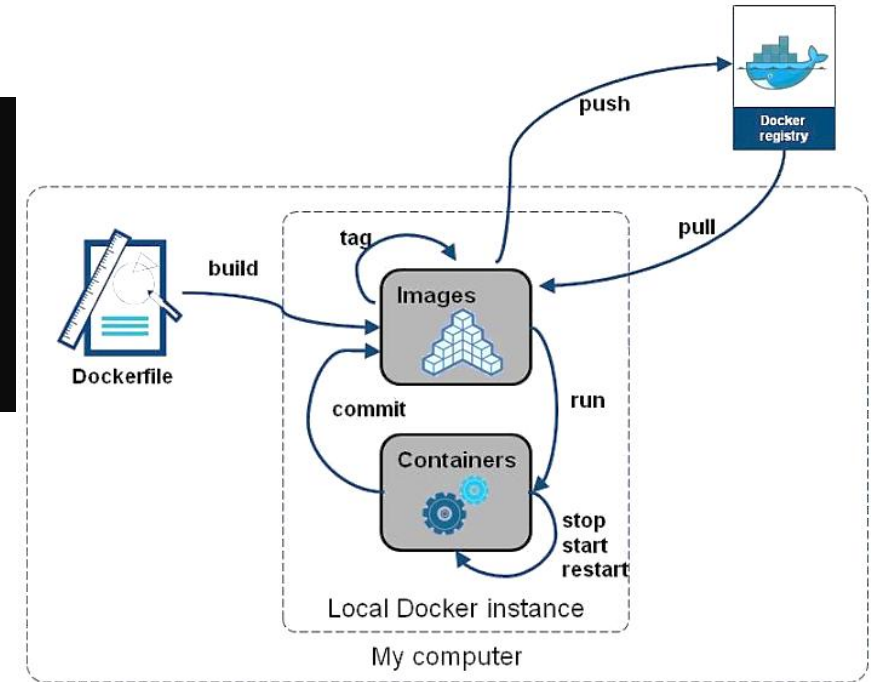
■ Voici les commandes les plus utilisées

- **Docker commit** : sert à créer une nouvelle image à partir des **modifications** apportées à un conteneur en cours d'exécution.

Modification
du conteneur

```
C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>docker run -d -p 8180:8080 --name myfirstapp firstapp
238d7a067d8372097f78f9df13e3e5392c57bcb6030c51bea4451dd9af92e67b

C:\Users\PC\Desktop\docker\node-bulletin-board-master\bulletin-board-app>docker commit myfirstapp
sha256:fae62802e60365ea682c1da24bccdffa01e86cc7f1b9b1d833f786cc0eaa8753
```



Conteneur - image

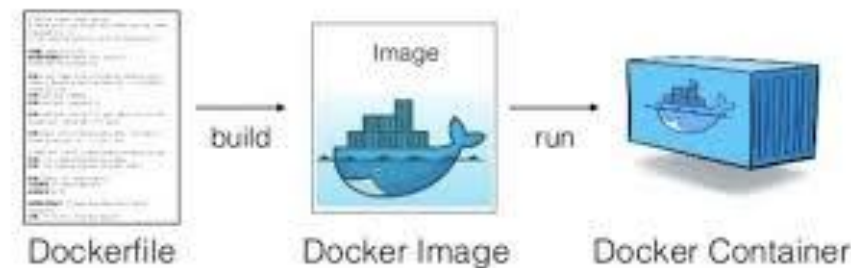
- Une **image** Docker est un package exécutable (sorte de Template prête à l'emploi) qui contient toutes les dépendances et les fichiers nécessaires pour exécuter une application.
- Les images Docker sont créées à partir d'un fichier de configuration appelé **Dockerfile**.
- Un **conteneur** est une **instance** d'une **image** Docker qui peut être exécutée de manière isolée sur un système hôte.
- Chaque conteneur contient une **application** et toutes les dépendances nécessaires pour son exécution



Docker build

Docker pull

Docker run





Dockerfile & package.json

- Ces deux fichiers sont très importants
- Un **Dockerfile** est un fichier de configuration qui contient les instructions nécessaires pour créer une image Docker.
- Ces instructions décrivent les actions que l'image doit exécuter une fois qu'elle sera créée.
- Elles seront lues une à une, puis exécutées indépendamment les unes des autres pour obtenir l'image souhaitée.
- Un Dockerfile précise :
 - le système d'exploitation sur lequel sera basé le conteneur,
 - les langages, variables environnementales,
 - emplacements de fichiers,
 - ports réseaux
 - Il spécifie les dépendances définies dans le fichier package.json
 - et autres composants requis.

Dockerfile

■ Voici un exemple de Dockerfile, avec explications des instructions :

- **FROM**: Définit l'image de départ (l'OS de base), ici une image de base Debian 11
- **RUN** : pour exécuter une commande dans votre conteneur
- **ADD** afin de copier ou de télécharger des fichiers dans l'image
- **COPY** copie des fichiers depuis votre PC vers l'image.
- **WORKDIR** qui permet de modifier le répertoire courant; La commande est équivalente à une commande `cd` en ligne de commande; L'ensemble des commandes qui suivront seront toutes exécutées depuis le répertoire défini.
- **RUN npm install** : permet d'installer permet d'installer les dépendances du projet.
- **EXPOSE** permet d'indiquer le port sur lequel l'application est à l'écoute.
- **VOLUME** permet d'indiquer quel répertoire sera partagé avec le host et servir à stocker les logs persistants.
- **CMD** définit la commande par défaut que le conteneur va exécuter lorsqu'il démarre, ici le fichier package.json doit contenir « start ».

```
# Utiliser l'image officielle Node.js avec la version
FROM node:18-bullseye
# MAJ les paquets yq (gere du yaml) et nettoyer
#après l'installation
RUN apt-get update -yq && apt-get install -yq curl &&
apt-get clean
# Copier tous les fichiers dans le répertoire /app
ADD . /app/
WORKDIR /app
# Copier uniquement package.json et installer les
#dépendances via npm
COPY package.json .
RUN npm install
# Exposer le port de l'application
EXPOSE 2368
# Déclarer un volume pour les logs
VOLUME /app/logs
# Lancer l'application
CMD ["npm", "run", "start"]
```

package.json

■ Le fichier package.json, permet de :

- **Définir les dépendances** : Il liste toutes les bibliothèques (ex: Express, React) que Docker doit installer à l'intérieur de l'image.
- **Définir des scripts de démarrage**: il définit les scripts que Docker utilisera pour savoir quelle commande lancer au démarrage du conteneur.
- **Déclarer des métadonnées du projet**: il contient des informations importantes sur l'application comme: nom, version, auteurs, etc., nécessaire pour la documentation et l'organisation du projet.
- **Automatiser** : il permet d'automatiser des tâches liées à Docker avec des scripts personnalisés comme la création et l'exécution.
 - ✓ Au lieu de taper « docker build -t firstapp . », il suffit taper : « npm run docker:build »

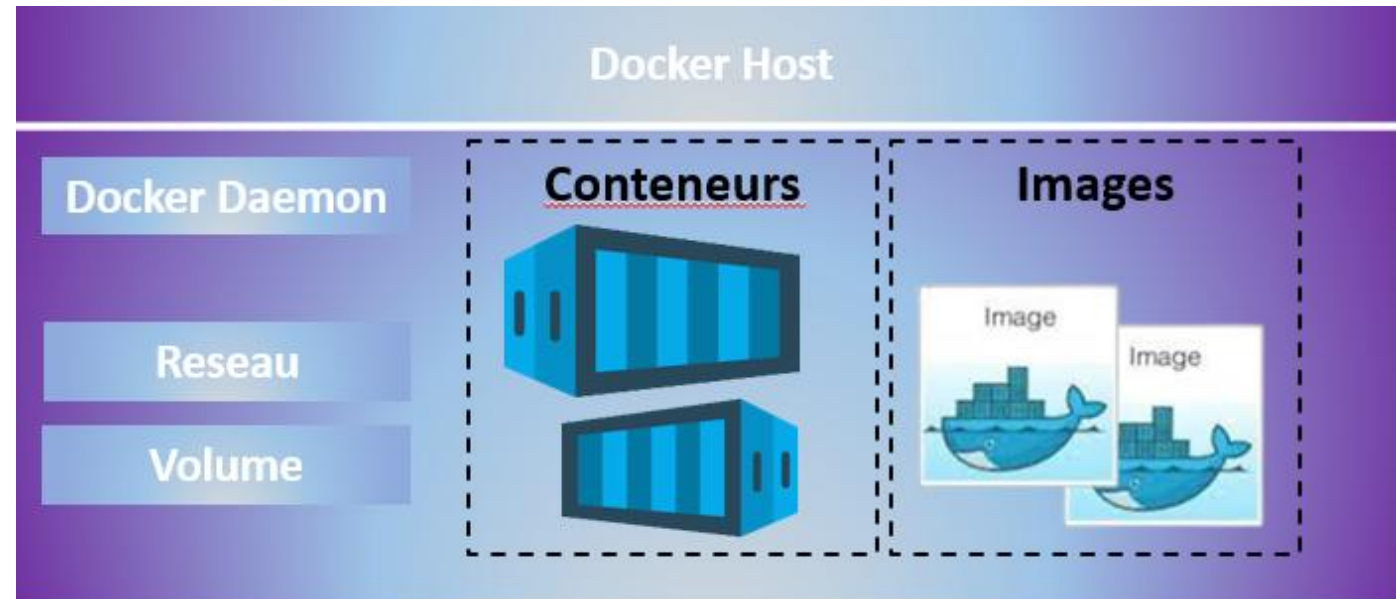
```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "Une application Node.js contene",
  "author": "DSBD",
  "license": "FSBM",
  "scripts": {
    "start": "node app.js",
    "docker:build": "docker build -t firstapp .",
  },

  "dependencies": {
    "bootstrap": "^5.3.3",
    "ejs": "^3.1.10",
    "express": "^4.21.2",
    "morgan": "^1.10.0",
    "vue": "^3.5.13",
    "vue-resource": "^1.5.3"},
}
```

Docker Host

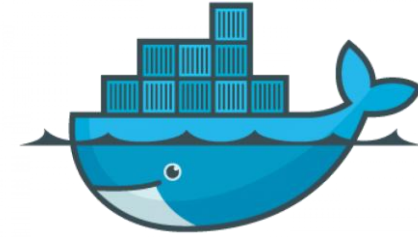


- Le Docker Host représente la machine physique ou virtuelle qui exécute le moteur Docker.
- Il fournit l'environnement et les ressources nécessaires pour créer, exécuter et gérer des conteneurs
 - Il assure aussi des fonctionnalités comme l'installation, la suppression, la mise à jour et la surveillance des conteneurs.
 - Plusieurs conteneurs Docker peuvent être exécutés simultanément sur un seul Docker host
- **Démon Docker** : processus d'arrière-plan chargé de recevoir les commandes et de les transmettre aux conteneurs.

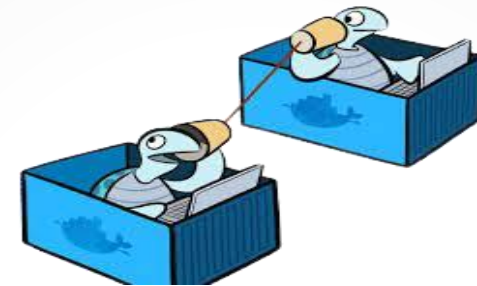


Docker Host

- **Réseaux** : Les réseaux Docker sont des réseaux virtuels qui assure la communication entre les conteneurs isolés.
- Il existe principalement **cinq pilotes** réseau dans le docker:
 - **bridge**: Les conteneurs ne peuvent communiquer qu'entre eux, donc ils ne sont pas accessibles de l'extérieur.
 - **none**: interdit toute communication interne ou externe avec le conteneur car ce dernier n'aura pas d'interface réseau.
 - **host**: permet aux conteneurs d'utiliser la même interface réseau que l'hôte. Dans ce cas, les conteneurs sont accessibles de l'extérieur et ne seront pas isolés les uns des autres dans le réseau.
 - **overlay**: ce mode est utilisé dans le cas de mise en réseau multi-hôte, dans ce cas les conteneurs s'exécutent sur différents hôtes Docker
 - **macvlan**: idéal lors de l'utilisation des applications nécessitant une connexion directe au réseau physique. Le macvlan attribue une adresse mac a un conteneur pour que celui-ci apparaisse comme un périphérique physique sur le réseau.



Réseau



Le système réseau de Docker utilise des drivers (pilotes). Plusieurs drivers existent et fournissent des fonctionnalités différentes.

Bridge

None

Host

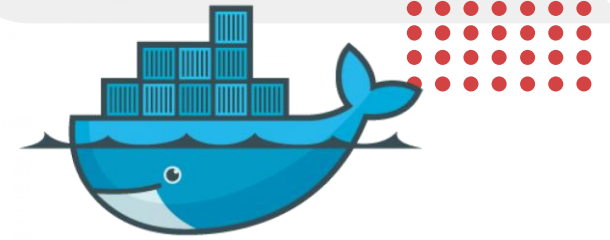
Overlay

Macvlan

Dans Docker, chaque conteneur possède sa propre interface réseau virtuelle qui est connectée à un réseau spécifique.

Docker Host

- **Volumes** : Les volumes Docker permettent de stocker des données **persistantes** pour les conteneurs Docker.
- Ils peuvent **être montés dans** des conteneurs pour stocker des données telles que des fichiers de configuration, des bases de données, des fichiers de log, etc.
- Les volumes sont **indépendants** des conteneurs, ils peuvent être utilisés pour stocker des données même si le conteneur est supprimé
- Cette approche permet d'éviter de sauvegarder **les données sur un conteneur**.



Volume



Créer

• Docker volume create

Utiliser

• Docker run -v [nom_volume]:[container_directory]

Etablir la liste des volumes

• Docker volume ls

Inspecter

• Docker volume inspect

Supprimer

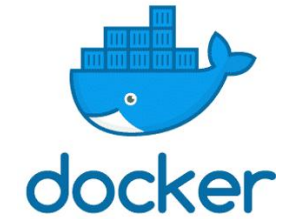
• Docker volume rm [volume_name]

Registry

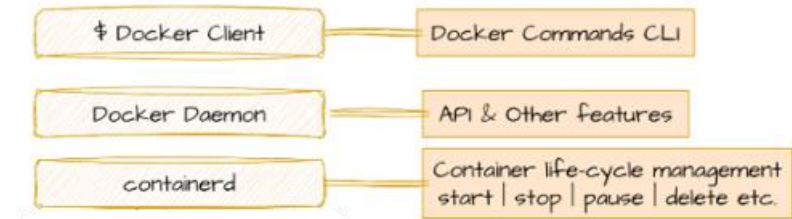
- Registry est un service qui permet stocker, gérer et distribuer des images de conteneurs (dépôt d'images).
- Il Fournit un emplacement centralisé pour que les développeurs publient les images de conteneurs qui seront accessible pour les utilisateurs à travers un API.
- Le registre public de Docker est appelé **Docker Hub**, mais il existe également des registres privés pour les entreprises.
- Docker Hub offre plusieurs services :
 - L'hébergement d'images Docker
 - Sécurité : Garantie que l'image téléchargée est bien celle qui a été publiée, sans altération
 - Intégration avec GitHub, etc.



Docker Engine

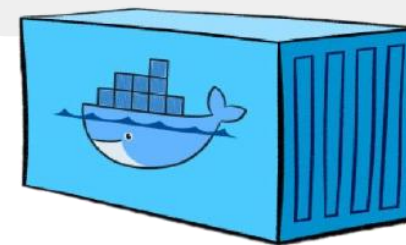


- **Docker Engine** est le composant principal de Docker qui permet de créer, de gérer et d'exécuter des conteneurs Docker sur un système hôte.
- Il est conçu pour fonctionner de manière invisible en arrière-plan,
- Cela inclut les composants suivants :
 - Le Docker Daemon (ou dockerd), qui est le processus principal du Docker Engine.
 - L'API REST, qui permet aux applications d'interagir avec Docker.
 - L'interface en ligne de commande Docker (CLI), qui permet aux utilisateurs d'interagir avec le Daemon via des commandes dans le terminal.



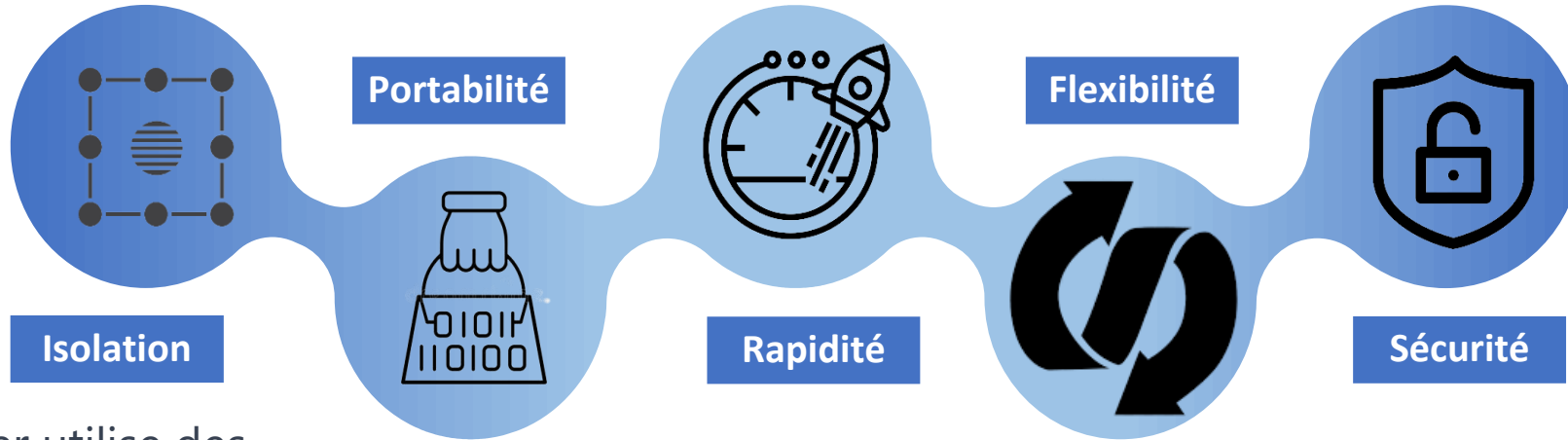
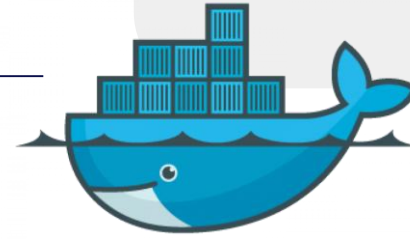
containerd est un **runtime** de conteneur, c'est un composant de bas niveau responsable de l'exécution des conteneurs.

Autres Objets Docker



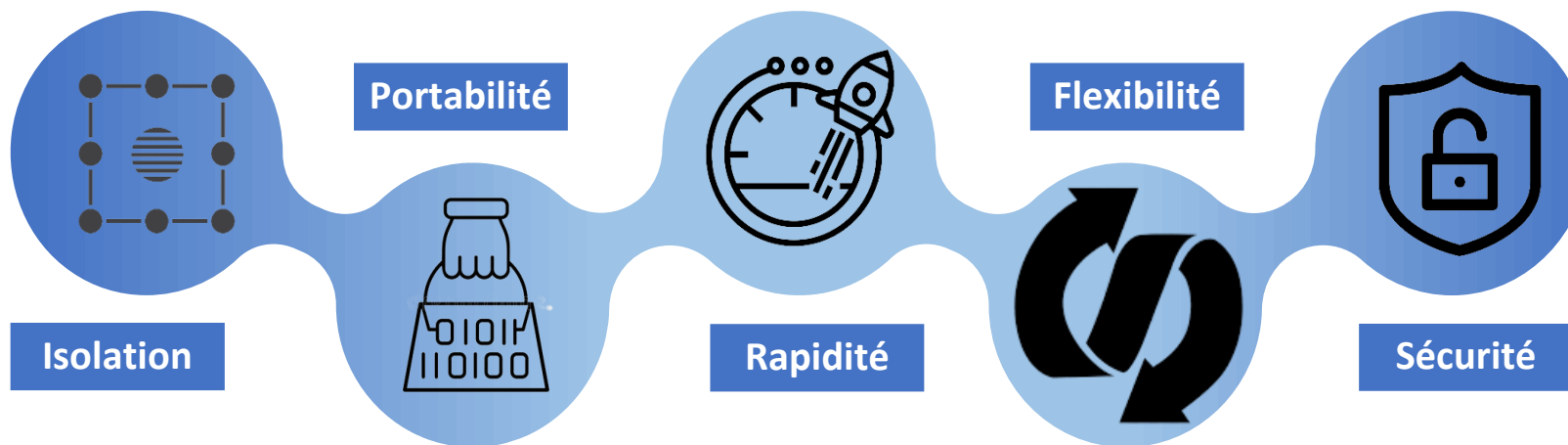
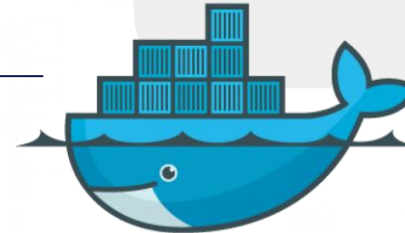
- **Docker Compose** : Docker Compose est un outil qui permet la gestion de multi-conteneurs sur **une seule machine**.
 - Il permet de définir un environnement de développement multi-conteneurs dans un fichier YAML (docker-compose.yml) et de déployer ces conteneurs avec une seule commande
- **Docker Swarm** : Docker Swarm est un outil d'orchestration de conteneurs.
 - Il permet de gérer des clusters de machines Docker et de déployer des applications sur **plusieurs hôtes Docker** de manière distribuée et hautement disponible

Avantages



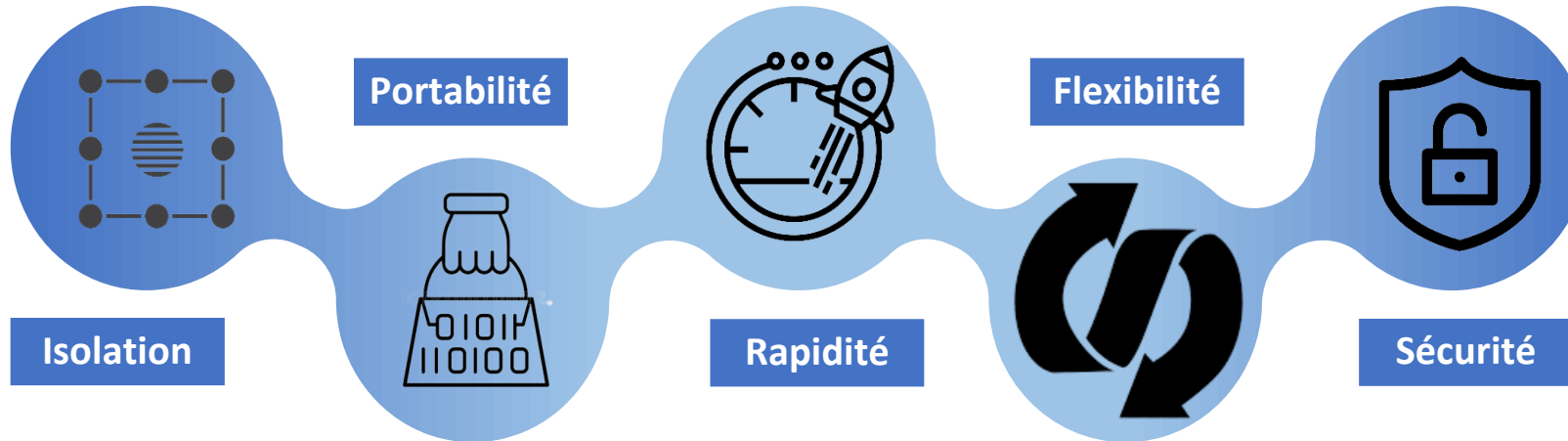
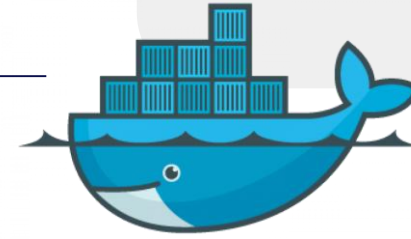
Isolation : Docker utilise des conteneurs pour isoler les applications et leurs dépendances, ce qui permet d'éviter les conflits entre les différentes applications et de garantir que chaque application fonctionne de manière prévisible.

Avantages



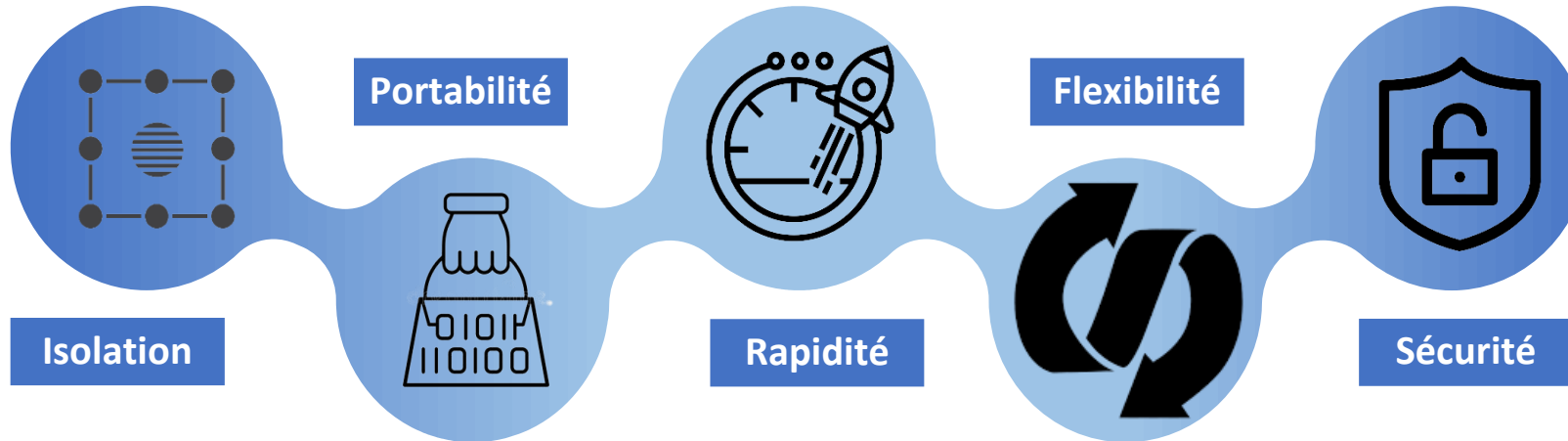
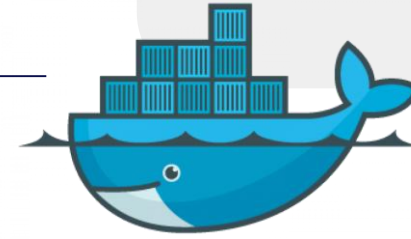
Portabilité : Les conteneurs Docker sont conçus pour fonctionner **sur n'importe quel système d'exploitation**, ce qui permet aux développeurs de créer une application une fois, puis de la déployer facilement sur différents environnements.

Avantages



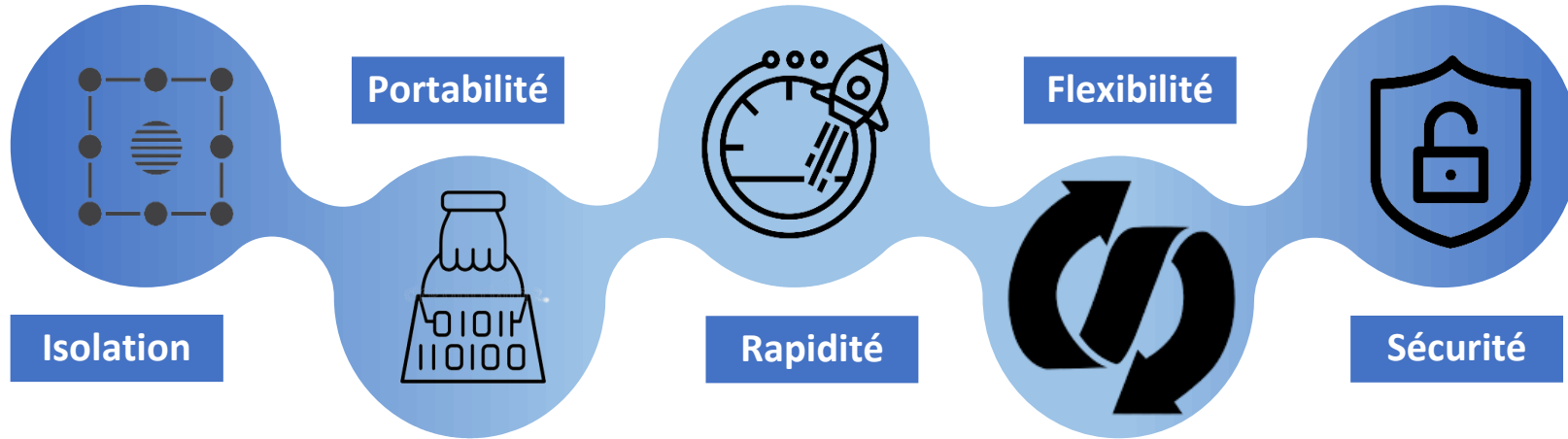
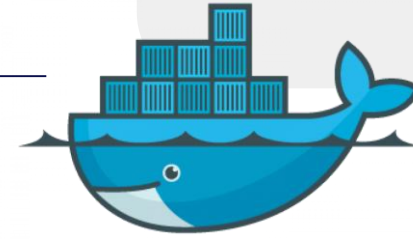
Rapidité : Les conteneurs Docker sont extrêmement légers et peuvent être **démarrés en quelques secondes**, ce qui permet aux développeurs de tester et de déployer rapidement des applications.

Avantages



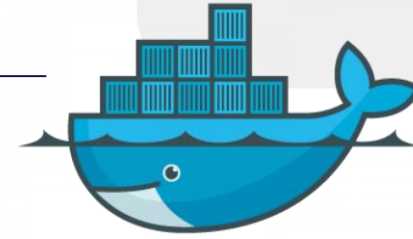
Flexibilité : Docker permet aux développeurs de créer des **microservices**, qui sont de petites applications indépendantes qui peuvent être combinées pour créer des applications plus complexes.

Avantages

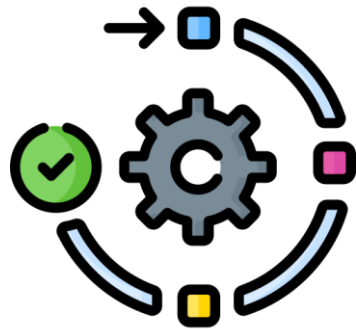


Sécurité : Docker utilise des mécanismes de sécurité intégrés pour garantir que les conteneurs sont **isolés** les uns des autres et que les applications sont protégées contre les menaces externes.

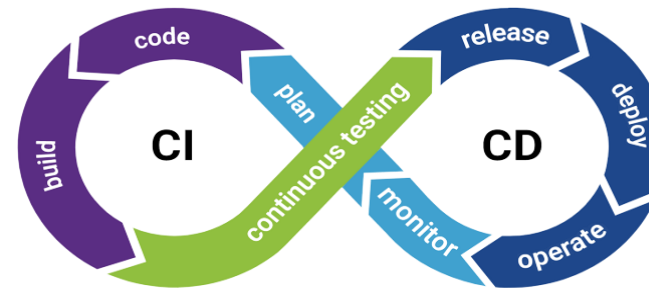
Docker et CCloud



Le Cloud Computing et Docker sont deux technologies qui peuvent être combinées pour fournir des environnements de développement et de déploiement flexibles et évolutifs.



Orchestration des conteneurs



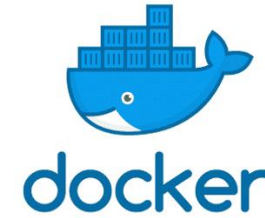
**Intégration continue CI et Déploiement continu
CD**



Inconvénient

- La plateforme Docker présente quelques inconvénients.
 - **Performance:** Il peut être difficile de gérer de façon efficace un grand nombre de conteneurs simultanément, et les conteneurs Docker peuvent présenter une surcharge de performance vu qu'ils s'exécutent sur le même Host.
 - **Sécurité :** Les containers sont isolés, mais partagent le même système d'exploitation.
 - ✓ Une attaque ou une faille de sécurité sur l'OS peut compromettre tous les conteneurs.
 - ✓ Les images de conteneurs peuvent contenir des vulnérabilités si elles ne sont pas gérées correctement, qui peuvent être exploitées pour accéder au host ou aux autres conteneurs.
 - **Taille des images :** Les images de conteneur Docker peuvent être de taille très importante et occuper une taille disque très grande.
 - **Dépendances :** Les conteneurs Docker peuvent avoir des dépendances qui doivent être mises à jour régulièrement.

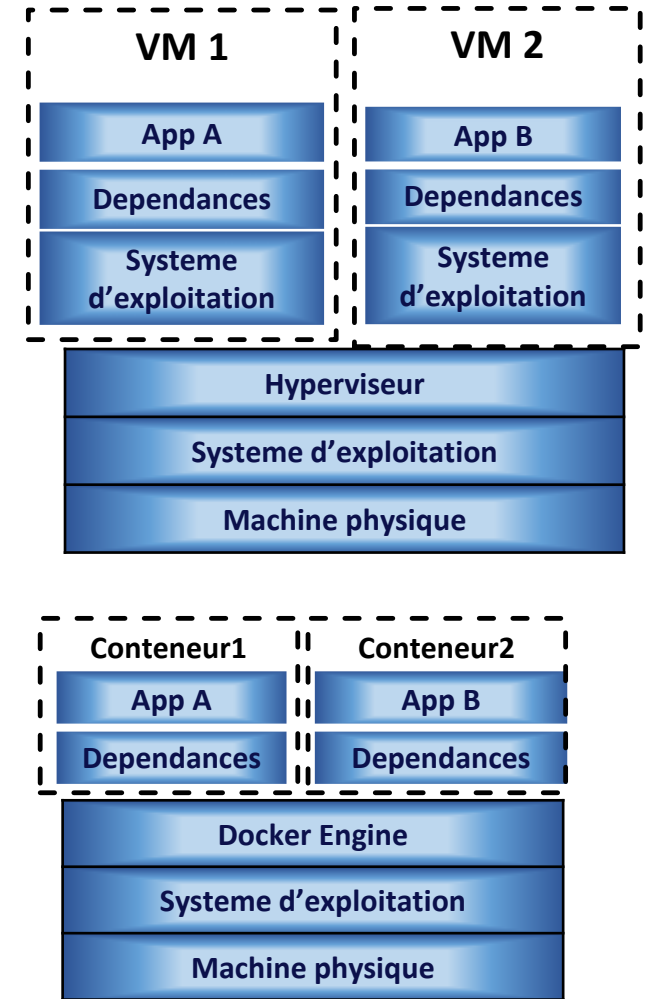
OpenVZ vs Docker



- OpenVZ voit un conteneur comme un vrai **serveur virtuel VPS** et Docker voit un conteneur comme une application ou un service.
- Les conteneurs Docker **ne sont pas strictement liés à la même version du noyau** que le système hôte
- Les VPS d'OpenVZ et les conteneurs LXC doivent avoir la même version du noyau que celle du système hôte.
- Les conteneur Docker fonctionnent sur n'importe quel **hôte Docker** indépendamment du système d'exploitation et de la configuration matérielle.
- Aujourd'hui, le moteur Docker repose sur des standards ouverts.
- Le moteur Docker utilise un format standardisé (OCI), ce qui le rend compatible avec la quasi-totalité de l'écosystème des conteneurs moderne.
- les conteneurs openVZ peuvent contenir n'importe quel nombre de processus
- Les conteneurs Docker contiennent généralement une application avec ses dépendances.

VM vs Docker

Caractéristique	Machines Virtuelles (VM)	Docker
Système d'exploitation	Chaque VM possède son propre OS complet (Guest OS)	Partagent le noyau (Kernel) de l'OS hôte.
Taille	Lourde	Très légère
Temps de démarrage	Lent (minutes)	Quasi instantané (secondes).
Utilisation CPU/RAM	Élevée (l'OS invité)	Faible (uniquement pour l'application).
Isolation	Maximale (totale au niveau matériel)	Partielle (isolation processus).
Portabilité	Moyenne (dépend de l'hyperviseur)	Excellente (tourne partout avec Docker).
Usage Type	Applications anciennes, isolation stricte	Microservices, Cloud, DevOps, CI/CD.





Virtualisation d'application basée sur le cloud

- SaaS: La virtualisation des applications basée sur le cloud est une approche de virtualisation dans laquelle les applications sont encapsulées dans des conteneurs et exécutées sur une infrastructure cloud.
- Les applications sont installées à distance sur un serveur virtuel ou physique au sein du datacenter
- Elles sont fournies comme service à la demande
- Des exemples de plateformes de ce genre il y a Amazon Web Services, Microsoft Azure et Google Cloud Platform .